Roubo ou furto?

Classificação de ocorrências de celulares subtraídos em São Paulo com uma rede MLP

Objetivo: dado um boletim de ocorrência já registrado (ou hipotético), estimar se ele é mais compatível com roubo ou furto e disponibilizar o modelo em uma aplicação web. A MLP não prevê a ocorrência de crimes.

Integrantes: [nome 1] · [nome 2] · [nome 3]
Global Solution – Aprendizado Profundo com Redes Neurais Convolucionais

Problema, fonte e recorte

Fonte: SSP-SP, Portal de Estatísticas → Bases de Dados → Celulares subtraídos (2025)
https://www.ssp.sp.gov.br/estatistica/consultas

Etapa do recorte	Registros
registros na base (estado de SP)	383.635
fatos ocorridos em São Paulo (capital)	216.129
rubrica = furto ou roubo	183.798
sem boletins ambíguos (furto+roubo)	183.657
1 linha por boletim (sem duplicidades)	161.570
datas consistentes (2024-2025, registro >= fato)	161.354
amostra utilizada	**20.000**

- Local do fato: CIDADE = S.PAULO (não o local de registro do BO)
- Rótulo: rubrica Roubo (art. 157) = 1 · Furto (art. 155) = 0
- Atributos: bairro, tipo e subtipo de local, período/hora, dia, mês, marca, dias até o registro, quantidade
- Excluídos por vazamento: rubrica, conduta, circunstância, desdobramento, flagrante, autoria, forma de apresentação
- Sem dados pessoais: logradouro, número, CEP, coordenadas

Preparação dos dados

Variável	Tratamento
Bairro, tipo/subtipo de local, marca	One-hot; categorias raras → "Outros" (40 bairros mais frequentes)
Período do dia	One-hot (hora exata quando existe; senão, campo DESCR_PERIODO)
Dia da semana	One-hot (7)
Mês e hora	Cíclico: seno e cosseno (dez≈jan, 23h≈0h) + flag "hora informada"
Dias até o registro	log(1+x) e z-score (média/desvio só do treino)
Quantidade	Limitada a 3 e escalada 0–1

102
atributos de entrada

14.000
treino (70%)

3.000
validação (15%)

3.000
teste (15%) — uso único

Divisão estratificada (seed 42). Sem hora exata → hora=0 + flag. Escalonamento e categorias ajustados só no treino (sem vazamento).

Arquitetura da MLP escolhida: M3 (128-64-32 +DO)



- Cada neurônio calcula z = w·x + b e aplica uma ativação; as camadas ocultas aprendem combinações dos atributos (ex.: tipo de local × período × marca).
- ReLU nas ocultas: simples e evita saturação do gradiente. Sigmoide na saída: transforma z em probabilidade entre 0 e 1 (limiar 0,5).
- Dropout (0,3): desliga aleatoriamente 30% dos neurônios durante o treino, reduzindo overfitting.
- Total de 23.553 parâmetros treináveis.

Configuração do treinamento

Item	Escolha
Função de perda	Binary cross-entropy (saída probabilística, 2 classes)
Otimizador	Adam, learning rate 1e-3 (5e-4 no M4)
Batch size	256 (128 no M4)
Épocas	Até 80, com early stopping (paciência 8 em val_loss, restaura os melhores pesos)
Métricas	Acurácia, precision, recall e F1 da classe roubo
Limiar de decisão	0,5
Reprodutibilidade	Seed 42; TensorFlow/Keras; pré-processamento ajustado só no treino

Por que BCE + sigmoide? É a combinação natural para classificação binária: a perda penaliza fortemente previsões confiantes e erradas.

Quatro arquiteturas testadas

Modelo	Estrutura	Por que mudamos	Params	Épocas *
M1 (16)	1 camada, 16 neurônios	Baseline simples	1.665	28
M2 (64-32)	2 camadas, 64-32	Mais capacidade	8.705	7
M3 (128-64-32 +DO)	3 camadas, 128-64-32 + Dropout 0,3	Mais profundidade + regularização	23.553	8
M4 (128-64 +L2+DO)	2 camadas, 128-64 + L2 1e-4 + Dropout 0,3; LR 5e-4; batch 128	Regularização dupla e LR menor	21.505	20

* época com menor perda de validação (early stopping).
Efeito observado: aumentar a rede quase não alterou o desempenho; o M2 atinge a melhor perda de validação já nas primeiras épocas (começa a sobreajustar); o Dropout (M3) deu a melhor perda e o melhor F1 de validação; L2 + LR menor (M4) não trouxe ganho.

Comparação e escolha do modelo

Validação (3.000 registros) — critério de seleção

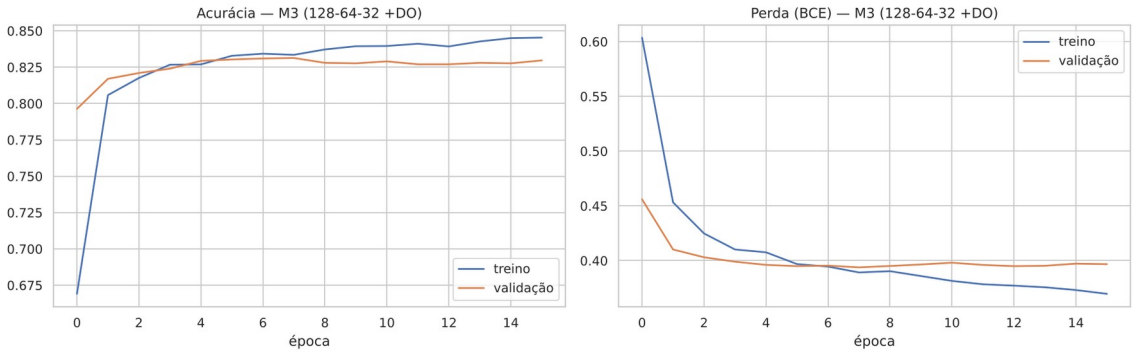
Modelo	Acurácia	Precision	Recall	F1	Val loss
M1 (16)	82.8%	0.742	0.824	0.781	0.394
M2 (64-32)	82.9%	0.737	0.840	0.785	0.397
M3 (128-64-32 +DO)	**83.1%**	**0.745**	**0.833**	**0.786**	**0.394**
M4 (128-64 +L2+DO)	82.9%	0.736	0.842	0.785	0.408

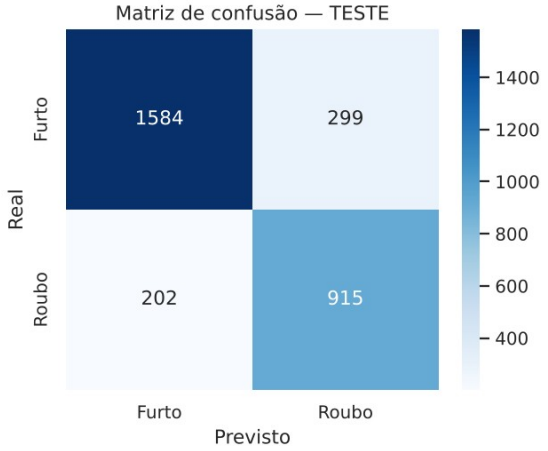
Teste final (uso único) — modelo escolhido vs. referências

Modelo	Acurácia	Precision	Recall	F1
M3 (128-64-32 +DO)	**83.3%**	**0.754**	**0.819**	**0.785**
Regressão logística (referência)	83.4%	0.761	0.808	0.784
Sempre "furto" (chute)	62.8%	–	0,000	0,000

Escolha: M3 (maior F1 e menor perda na validação). As diferenças entre modelos são pequenas, dentro do ruído; a regressão logística empata com a MLP.

Curvas, matriz de confusão e diagnóstico

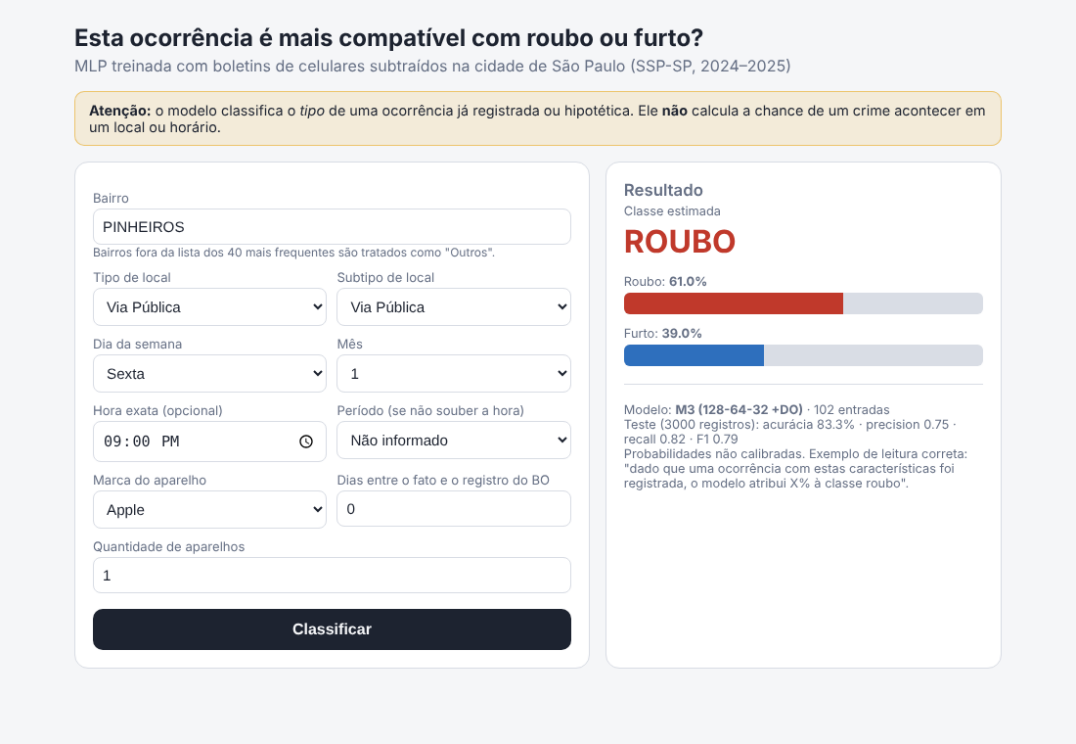




- Treino ≈ validação em todos os modelos (gap < 2 p.p.): sem overfitting severo; leve sinal no M2 e no M4.
- Sem underfitting relevante: o M1 (16 neurônios) já chega perto das redes maiores.
- Teste: 915 roubos acertados, 202 roubos classificados como furto, 299 furtos classificados como roubo, 1584 furtos acertados.
- O teto de desempenho vem da informação disponível nos atributos, não do tamanho da rede.

Matriz no conjunto de teste (3.000 registros).

Aplicação web



- Flask + NumPy: o servidor carrega os pesos da MLP treinada (exportados do Keras) e reproduz a rede; diferença máxima vs. Keras < 1e-7.
- Formulário com exatamente os atributos usados no treino; mesmo pré-processamento (src/features.py).
- Mostra a classe estimada e as probabilidades de roubo e furto.
- Aviso fixo: o modelo classifica o tipo de uma ocorrência, não o risco de crime.
- Link: [colar URL do deploy] · Código: [colar URL do GitHub]

Conclusões, limitações e uso de IA generativa

Conclusões

- A MLP acerta 83.3% no teste (chute: 62.8%) com F1 de 0.79 para roubo.
- Mais camadas ≠ mais acerto: as 4 arquiteturas e a regressão logística ficam muito próximas.
- A informação dos atributos, e não a rede, limita o resultado.

Limitações

- Dados administrativos: subnotificação e erros de preenchimento.
- Só celulares, 2024–2025; ~50% sem hora exata.
- Divisão aleatória (sem validação temporal); probabilidades não calibradas.
- Não usar para decisões sobre pessoas ou policiamento.

IA generativa (Claude)
Apoio em exploração da base, código de pré-processamento, treino e app, e rascunho da documentação. A equipe revisou o código, checou vazamento de dados e valida e explica os resultados.